API Endpoint Design

University ERP · Advanced Software Engineering · UCSC

1. Overview

This document defines the REST API endpoints for all eight University ERP microservices. All endpoints follow REST conventions: nouns not verbs in paths, correct HTTP verbs, and standard HTTP status codes. All request and response bodies use JSON. All endpoints (except /health and auth register/login) require a valid JWT in the Authorization: Bearer <token> header.

1.1 Conventions

- Base URL per service: http://<host>:<port> (routed through NGINX at /api/<resource>)
- All request bodies: Content-Type: application/json
- All responses: Content-Type: application/json
- Authentication: Authorization: Bearer <jwt> (except POST /api/auth/login and /register)
- :id refers to MongoDB ObjectId unless stated otherwise
- Timestamps are ISO 8601 strings (e.g. 2025-07-11T10:00:00.000Z)

1.2 Standard Status Codes

Code	Meaning	When used
200 OK	Success	Successful GET, PUT
201 Created	Resource created	Successful POST that creates a resource
400 Bad Request	Validation error	Missing required fields, invalid format
401 Unauthorized	Auth failed	Missing or invalid JWT
403 Forbidden	Insufficient role	Authenticated user lacks the required role for this endpoint
404 Not Found	Resource missing	ID does not exist in this service's DB
409 Conflict	Duplicate resource	Unique constraint violation (e.g. duplicate email)
502 Bad Gateway	Upstream failure	A cross-service REST call failed
500 Internal Server Error	Unexpected error	Unhandled exception

1.3 Standard Error Response

All 4xx and 5xx responses return a JSON body in this format:

```
{ "error": "Human-readable description of the problem" }
```

For upstream service failures (502), an optional detail field is included:

```
{ "error": "Student Service unavailable",
  "detail": "connect ECONNREFUSED 172.18.0.3:3001" }
```

Error Code Reference:

HTTP Code	Meaning	Example error value
400 Bad Request	Validation failed / bad input	"name is required"
401 Unauthorized	Missing or invalid JWT token	"No token provided" / "Invalid token"
403 Forbidden	Authenticated but insufficient role	"Forbidden"
404 Not Found	Resource not found	"Student not found"
409 Conflict	Unique constraint violated	"Email or studentId already exists"
502 Bad Gateway	Upstream service unavailable	"Student Service unavailable"
500 Internal Server Error	Unexpected server error	"Internal server error"

1.4 Role-Based Access Control

All authenticated endpoints enforce role-based access. The five roles defined in the Auth Service are:

Role	Description
ADMIN	Full access — create users, manage all data, bulk import, assign specializations
EXAM_DIVISION	Manage exams, results, registration windows, approve/reject entries, upload results
HOD	Approve or reject exam entries; read-only access to all student, exam, and result data
LECTURER	Approve or reject exam entries; read-only access to results
STUDENT	View own enrolment and results; apply for specialization; upload own photo

The table below lists the permitted roles for each write/management endpoint. All GET endpoints are readable by every authenticated role unless stated otherwise.

Endpoint	Allowed Roles	Notes
POST /api/students	ADMIN, EXAM_DIVISION	Create student account
POST /api/students/bulk	ADMIN, EXAM_DIVISION	Bulk CSV import
PATCH /api/students/:id	ADMIN, EXAM_DIVISION (full); STUDENT (own, limited fields)	Cannot change studentId, email, nic, permanentDistrict
POST /api/students/:id/photo	ADMIN, EXAM_DIVISION	Profile photo upload
POST /api/students/:id/year-registration	ADMIN, EXAM_DIVISION	Academic year enrolment
POST /api/exams	ADMIN, EXAM_DIVISION	Create exam
PATCH /api/exams/:id/schedule	ADMIN, EXAM_DIVISION	Set registration window dates
PATCH /api/exams/:id/toggle	ADMIN, EXAM_DIVISION	Open or close exam registration
PATCH /api/exams/:id/entries/:entryId	ADMIN, EXAM_DIVISION, HOD, LECTURER	Approve or reject an exam entry
POST /api/results	ADMIN, EXAM_DIVISION	Upload subject result
POST /api/results/bulk	ADMIN, EXAM_DIVISION	Bulk CSV result import
PATCH /api/results/gpa/:id/finalize	ADMIN, EXAM_DIVISION	Finalise year GPA record
PATCH /api/specializations/:id/assign	ADMIN, EXAM_DIVISION	Assign specialization to student
GET /api/notifications	ADMIN, EXAM_DIVISION, HOD, LECTURER	View Kafka event audit log
All GET endpoints	All authenticated roles	Read access for every authenticated user

2. Auth Service (port 3007 · auth_db)

Method	Path	Auth	Description
POST	/api/auth/register	✗	Register a new user account
POST	/api/auth/login	✗	Login and receive a JWT
POST	/api/auth/refresh	✓	Issue a new JWT using a valid (non-expired) existing token
GET	/health	✗	Health check — returns service status

POST /api/auth/register

Request Body:

```
{
  "name": "string (required)",
  "email": "string (required, unique)",
  "password": "string (required, min 8 chars)",
  "role": "student | admin (default: student)"
}
```

Response:

```
{
  "_id": "ObjectId",
  "name": "string",
  "email": "string",
  "role": "string",
  "createdAt": "ISO8601"
}
```

Status codes: 201, 400, 409

Note: Password is hashed with bcrypt (work factor 12) before storage. Plain password never stored.

POST /api/auth/login

Request Body:

```
{
  "email": "string (required)",
  "password": "string (required)"
}
```

Response:

```
{
  "token": "JWT string (expires 24h)",
  "user": { "_id": "...", "name": "...", "role": "..." }
}
```

Status codes: 200, 400, 401

*Note: Token is signed with JWT_SECRET. Payload contains { userId, role, iat, exp }.*POST /api/auth/refresh

Response:

```
{
  "token": "new JWT string"
}
```

Status codes: 200, 401

GET /health

Response:

```
{ "status": "ok", "service": "auth-service" }
```

Status codes: 200

3. Student Service (port 3001 · students_db)

Method	Path	Auth	Description
POST	/api/students	✓	Create a new student record
GET	/api/students	✓	List all students (sorted by createdAt descending)
GET	/api/students/:id	✓	Get a single student by MongoDB _id
PUT	/api/students/:id	✓	Update a student record (partial update supported)
GET	/health	✗	Health check

POST /api/students

Request Body:

```
{
  "name": "string (required)",
  "email": "string (required, unique)",
  "studentId": "string (required, unique)",
  "programme": "string (optional)"
}
```

Response:

```
{
  "_id": "ObjectId",
  "name": "string",
  "email": "string",
  "studentId": "string",
  "programme": "string",
  "createdAt": "ISO8601",
  "updatedAt": "ISO8601"
}
```

Status codes: 201, 400, 401, 409

*Note: Publishes **student.created** Kafka event on success (designed; not wired in prototype).*

GET /api/students

Response:

```
[
  { "_id": "...", "name": "...", "email": "...", "studentId": "...", ... },
  ...
]
```

Status codes: 200, 401

GET /api/students/:id

Response:

```
{
  "_id": "ObjectId",
  "name": "string",
  "email": "string",
  "studentId": "string",
  "programme": "string",
  "createdAt": "ISO8601"
}
```

Status codes: 200, 401, 404

Note: Called internally by Exam Service to verify a student before exam enrolment.

PUT /api/students/:id

Request Body:

```
{
  "name": "string (optional)",
  "programme": "string (optional)"
}
```

Response:

```
{
  "_id": "...", "name": "...", "updatedAt": "ISO8601", ...
}
```

Status codes: 200, 400, 401, 404

Note: email and studentId are immutable after creation.

GET /health

Response:

```
{ "status": "ok", "service": "student-service" }
```

Status codes: 200

4. Course Service (port 3002 · courses_db)

Method	Path	Auth	Description
POST	/api/courses	✓	Create a new course
GET	/api/courses	✓	List all courses
GET	/api/courses/:id	✓	Get a single course by ID
POST	/api/courses/:id/modules	✓	Add a module to a course
GET	/api/courses/:id/modules	✓	List all modules for a course (sorted by week)

POST /api/courses

Request Body:

```
{
  "title": "string (required)",
  "courseCode": "string (required, unique)",
  "credits": "number (required)"
}
```

Response:

```
{
  "_id": "ObjectId",
  "title": "string",
  "courseCode": "string",
  "credits": "number",
  "createdAt": "ISO8601"
}
```

Status codes: 201, 400, 401, 409

GET /api/courses

Response:

```
[ { "_id": "...", "title": "...", "courseCode": "...", "credits": 3 }, ... ]
```

Status codes: 200, 401

GET /api/courses/:id

Response:

```
{ "_id": "...", "title": "...", "courseCode": "...", "credits": 3 }
```

Status codes: 200, 401, 404

POST /api/courses/:id/modules

Request Body:

```
{
  "title": "string (required)",
  "week": "number (required)"
}
```

Response:

```
{
  "_id": "ObjectId",
  "courseId": "ObjectId",
  "title": "string",
  "week": "number"
}
```

Status codes: 201, 400, 401, 404

GET /api/courses/:id/modules

Response:

```
[ { "_id": "...", "courseId": "...", "title": "...", "week": 1 }, ... ]
```

Status codes: 200, 401, 404

5. Exam Service (port 3003 · exams_db)

Method	Path	Auth	Description
POST	/api/exams	✓	Create a new exam
GET	/api/exams	✓	List all exams (sorted by date ascending)
POST	/api/exams/:id/entries	✓	Enrol a student in an exam — verifies student via Student Service
GET	/api/exams/:id/entries	✓	List all student entries for an exam
DELETE	/api/exams/:id/entries/:entryId	✓	Remove a student entry from an exam
GET	/health	✗	Health check

POST /api/exams

Request Body:

```
{
  "title": "string (required)",
  "courseId": "string (required)",
  "date": "ISO8601 (required)",
  "venue": "string (optional)"
}
```

Response:

```
{
  "_id": "ObjectId",
  "title": "string",
  "courseId": "string",
  "date": "ISO8601",
  "venue": "string",
  "createdAt": "ISO8601"
}
```

Status codes: 201, 400, 401

GET /api/exams

Response:

```
[ { "_id": "...", "title": "...", "courseId": "...", "date": "..." }, ... ]
```

Status codes: 200, 401

POST /api/exams/:id/entries

Request Body:

```
{
  "studentId": "MongoDB ObjectId of the student (required)"
}
```

Response:

```
{
  "_id": "ObjectId",
  "examId": "ObjectId",
  "studentId": "string",
  "studentName": "string",
  "enrolledAt": "ISO8601"
}
```

Status codes: 201, 400, 401, 404, 409, 502

Note: Makes internal GET /api/students/:id call to Student Service. 404 if student not found. 502 if Student Service is down.

GET /api/exams/:id/entries

Response:

```
{
  "exam": { "_id": "...", "title": "...", "date": "..." },
  "entries": [ { "studentId": "...", "studentName": "...", "enrolledAt": "..."
}, ... ]
}
```

Status codes: 200, 401, 404

DELETE /api/exams/:id/entries/:entryId

Response:

```
{ "message": "Entry removed" }
```

Status codes: 200, 401, 404

GET /health

Response:

```
{ "status": "ok", "service": "exam-service" }
```

Status codes: 200

6. Result Service (port 3004 · results_db)

Method	Path	Auth	Description
POST	/api/results	✓	Record a new exam result
GET	/api/results/student/:studentId	✓	Get all results for a specific student
GET	/api/results/exam/:examId	✓	Get all results for a specific exam
POST	/api/results/:id/publish	✓	Publish a result — sets publishedAt and emits **result.published** Kafka event
GET	/health	✗	Health check

POST /api/results

Request Body:

```
{
  "studentId": "string (required)",
  "examId": "string (required)",
  "score": "number 0-100 (required)",
  "grade": "A | B | C | D | F (required)"
}
```

Response:

```
{
  "_id": "ObjectId",
  "studentId": "string",
  "examId": "string",
  "score": "number",
  "grade": "string",
  "publishedAt": null,
  "createdAt": "ISO8601"
}
```

Status codes: 201, 400, 401, 409

GET /api/results/student/:studentId

Response:

```
[ { "_id": "...", "examId": "...", "score": 85, "grade": "A", ... }, ... ]
```

Status codes: 200, 401

Note: Called by Transcript Service when generating a PDF.

GET /api/results/exam/:examId

Response:

```
[ { "studentId": "...", "score": 72, "grade": "B", ... }, ... ]
```

Status codes: 200, 401

POST /api/results/:id/publish

Response:

```
{
  "_id": "...",
  "publishedAt": "ISO8601",
  ...
}
```

Status codes: 200, 401, 404

*Note: Emits **result.published** Kafka event with { studentId, examId, grade, score }.*

GET /health

Response:

```
{ "status": "ok", "service": "result-service" }
```

Status codes: 200

7. Transcript Service (port 3005 · transcripts_db)

Method	Path	Auth	Description
POST	/api/transcripts/:studentId/generate	✓	Generate a PDF transcript for a student
GET	/api/transcripts/:studentId	✓	Get transcript metadata for a student
GET	/api/transcripts/:studentId/download	✓	Download the PDF transcript file

POST /api/transcripts/:studentId/generate

Response:

```
{
  "_id": "ObjectId",
  "studentId": "string",
  "generatedAt": "ISO8601",
  "pdfPath": "string (server file path)"
}
```

Status codes: 201, 401, 404, 502

Note: Calls GET /api/results/student/:studentId on Result Service to fetch grades. Returns 502 if Result Service is unavailable.

GET /api/transcripts/:student

Response:

```
{ "_id": "...", "studentId": "...", "generatedAt": "...", "pdfPath": "..." }
```

Status codes: 200, 401, 404

GET /api/transcripts/:studentId/download

Response:

```
application/pdf binary stream
```

Status codes: 200, 401, 404

Note: Response Content-Type is application/pdf, not JSON.

8. Notification Service (port 3006 · notif_db)

Method	Path	Auth	Description
GET	/api/notifications/:studentId	✓	Get notification audit log for a student
GET	/health	✗	Health check

GET /api/notifications/:studentId

Response:

```
[
  { "_id": "...", "studentId": "...", "event": "exam.registered", "sentAt":
"...", "channel": "email" },
  ...
]
```

Status codes: 200, 401

Note: Primarily a Kafka consumer. REST endpoint is for audit/admin only.

GET /health

Response:

```
{ "status": "ok", "service": "notification-service" }
```

Status codes: 200

9. Reporting Service (port 3008 · reports_db)

Method	Path	Auth	Description
GET	/api/reports/summary	✓	Get overall system metrics snapshot
GET	/api/reports/exam/:examId	✓	Get metric snapshot for a specific exam
GET	/api/reports/students	✓	Get student enrolment and performance statistics
GET	/health	✗	Health check

GET /api/reports/summary

Response:

```
{
  "totalStudents": "number",
  "totalExams": "number",
  "totalResults": "number",
  "averagePassRate": "number (0-100)",
  "generatedAt": "ISO8601"
}
```

Status codes: 200, 401

Note: Data is read from pre-computed snapshots in reports_db — not a live query.

GET /api/reports/exam/:examId

Response:

```
{
  "examId": "string",
  "totalEntries": "number",
  "passRate": "number",
  "gradeDistribution": { "A": 5, "B": 10, "C": 8, "D": 2, "F": 1 },
  "createdAt": "ISO8601"
}
```

Status codes: 200, 401, 404

GET /api/reports/students

Response:

```
{
  "totalEnrolled": "number",
  "byProgramme": { "CS": 40, "EE": 20 },
  "averageGrade": "string"
}
```

Status codes: 200, 401

GET /health

Response:

```
{ "status": "ok", "service": "reporting-service" }
```

Status codes: 200

10. Complete Endpoint Summary

All endpoints across all services at a glance.

Service	Method	Path	Auth	Status Codes
Auth	POST	/api/auth/register	✗	201,400,409
Auth	POST	/api/auth/login	✗	200,400,401
Auth	POST	/api/auth/refresh	✓	200,401
Student	POST	/api/students	✓	201,400,401,409
Student	GET	/api/students	✓	200,401
Student	GET	/api/students/:id	✓	200,401,404
Student	PUT	/api/students/:id	✓	200,400,401,404
Course	POST	/api/courses	✓	201,400,401,409
Course	GET	/api/courses	✓	200,401
Course	GET	/api/courses/:id	✓	200,401,404
Course	POST	/api/courses/:id/modules	✓	201,400,401,404
Course	GET	/api/courses/:id/modules	✓	200,401,404
Exam	POST	/api/exams	✓	201,400,401
Exam	GET	/api/exams	✓	200,401
Exam	POST	/api/exams/:id/entries	✓	201,400,401,404,409,502
Exam	GET	/api/exams/:id/entries	✓	200,401,404
Exam	DELETE	/api/exams/:id/entries/:entryId	✓	200,401,404
Result	POST	/api/results	✓	201,400,401,409
Result	GET	/api/results/student/:studentId	✓	200,401
Result	GET	/api/results/exam/:examId	✓	200,401
Result	POST	/api/results/:id/publish	✓	200,401,404
Transcript	POST	/api/transcripts/:studentId/generate	✓	201,401,404,502
Transcript	GET	/api/transcripts/:studentId	✓	200,401,404
Transcript	GET	/api/transcripts/:studentId/download	✓	200,401,404
Notification	GET	/api/notifications/:studentId	✓	200,401
Reporting	GET	/api/reports/summary	✓	200,401
Reporting	GET	/api/reports/exam/:examId	✓	200,401,404
Reporting	GET	/api/reports/students	✓	200,401

References: Service Identification Table, Bounded Context Diagram, Service Communication Diagram.